



جامعة أم القرى
UMM AL-QURA UNIVERSITY

College of Computing
Software Development and Operations — SE4206

DevOps Project Report

Secure CI/CD Pipeline with DevSecOps Gates

An automated GitHub Actions pipeline that scans every change for vulnerabilities, ships a health-checked Docker container, and alerts the team the moment anything breaks.

Team — Section 2

Jana Alyamani
Noura Alkabkabi
Amjad Hawsawi
Weaam Rashed

Supervised by Dr. Ghofran Talbh
Software Engineering Program · 2026

Table of Contents

1. Introduction	3
1.1 Project Objectives	3
2. Background: From DevOps to DevSecOps	3
3. System Architecture	4
3.1 End-to-End Flow	4
3.2 Roles in the Team	4
4. The CI/CD Pipeline (GitHub Actions)	5
4.1 The Trigger and Job Definition	5
4.2 The Ten Steps, in Order	5
5. Security Scanning (The Gate)	6
5.1 Severity Policy	6
5.2 Proving the Gate Works: One Dependency, Two Outcomes	6
6. Docker and the Health Check	8
6.1 The Dockerfile	8
6.2 Healthy vs. Unhealthy — The Same Image, Two Outcomes	8
7. Real-Time Notifications	9
7.1 The Notify Step	10
8. The Live Demo	10
Scenario A — Success (branch: main)	10
Scenario B — Failure (branch: demo/security-fail → main)	11
9. Challenges and Lessons Learned	12
10. Conclusion	12
11. References	12

1. Introduction

Modern software teams release new code almost every day. That speed is great for users, but it also creates a hidden risk: the third-party libraries we pull into a project can carry known security holes that an attacker may find before we do. In a traditional workflow, security is checked once, right before release — and by then a vulnerable dependency may already be sitting in production.

This project takes a different approach. We built a **Secure CI/CD Pipeline** that applies the **DevSecOps** idea of “shifting security left” — meaning security checks run automatically on every code change, not at the end. The pipeline scans dependencies, runs tests, builds the app into a Docker container, verifies that the container is actually healthy, and finally notifies the team in real time over Telegram. If any safety check fails, the pipeline stops immediately and an alert is sent.

The full project is hosted on GitHub at github.com/janalyamm/secure-cicd-devsecops-demo2.

1.1 Project Objectives

- Automate the full build, test, and packaging process so it runs without any manual clicks.
- Add an automatic security gate that blocks the pipeline when a dependency has a high or critical vulnerability.
- Containerize the application with Docker and confirm it is not just running, but actually *healthy*.
- Send instant Telegram alerts to the team for both successful and failed runs.
- Show, with a live demo, the difference between a passing run and a deliberately blocked one.

2. Background: From DevOps to DevSecOps

CI/CD stands for **Continuous Integration** and **Continuous Delivery**. Continuous Integration means every change is automatically built and tested the moment it is pushed, so problems appear in minutes instead of days. Continuous Delivery means that once the checks pass, the same automation packages the app into a ready-to-deploy artifact — in our case, a Docker image.

DevSecOps is CI/CD with security built into the pipeline itself, instead of treated as a separate, final step done by another team. The table below summarizes the difference between the traditional approach and the one we followed.

Dimension	Traditional DevOps	Our DevSecOps Approach
Step sequence	Build → Test → Deploy	Build → Scan → Test → Verify → Deploy
When security runs	Once, just before release	On every push and every PR
Where security sits	A separate team outside the pipeline	A built-in step inside the pipeline
Response style	Reactive — after the incident	Proactive — shift-left

The key term here is **“shift left”**: moving the security check earlier (to the left) in the timeline, so a vulnerability is caught at the pull-request stage and never reaches the main branch.

3. System Architecture

The pipeline is triggered whenever a developer pushes code or opens a pull request to the main branch. GitHub spins up a fresh ubuntu-latest runner, reads the workflow file, and executes the steps in order. Two safety gates sit inside this flow: a **security gate** (the dependency scan) and a **health gate** (the Docker health check). Secrets such as the Telegram bot token are injected securely and never written into the code.

3.1 End-to-End Flow

Stage	What happens
Developer	Pushes code or opens a pull request to main.
GitHub Repository	Receives the change and triggers the workflow.
Actions Runner	A clean ubuntu-latest machine runs all the steps.
Security Scan (Gate 1)	npm audit checks every dependency; high/critical findings stop the run.
Tests + Build	Project test and build scripts run.
Docker	The app is built into an image and started as a container.
Health Check (Gate 2)	The pipeline waits until the container reports healthy.
Notify	A Telegram message is sent — success or failure.

A core design principle is that the pipeline is **fail-closed**: a non-zero exit from either the security scan or the health check short-circuits the remaining work and routes the run to the failure-notification path. The container is always stopped afterwards (via an `always()` step) to avoid leaving runtime state on the runner.

3.2 Roles in the Team

The project was divided into four clear areas of ownership, with every member responsible for one part of the pipeline:

Member	Responsibility	Deliverables
Jana Alyamani	CI/CD & GitHub Actions	The workflow file, install/build/test stages, and the push & PR triggers.
Noura Alkabbabi	Security Scanning	The npm audit gate, severity policy, and the deliberately vulnerable dependency.

Member	Responsibility	Deliverables
Amjad Hawsawi	Docker & Health Check	The Dockerfile, the /health endpoint, and the healthy vs. unhealthy demo.
Weaam Rashed	Notifications & Demo	Telegram alerts, GitHub Secrets, architecture docs, and the live demo.

4. The CI/CD Pipeline (GitHub Actions)

The whole pipeline lives in a single workflow file, `.github/workflows/main.yml`. One trigger block makes the same job run on a direct push to main and on any pull request targeting it. The job runs on a clean runner with no leftover state from previous runs, and the steps execute top to bottom in a fail-fast manner — if one step exits non-zero, the rest are skipped and the run is marked as failed.

4.1 The Trigger and Job Definition

```
name: Secure CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  ci:
    runs-on: ubuntu-latest
```

4.2 The Ten Steps, in Order

#	Step	Command / Action
01	Checkout	actions/checkout@v4 — pull the repo code onto the runner
02	Setup Node	actions/setup-node@v4 — prepare Node 18
03	Install Deps	npm install — resolve dependencies
04	Security Scan (GATE)	npm run audit:ci — fail-closed on high/critical
05	Run Tests	npm test
06	Build Project	npm run build
07	Build Docker Image	docker build -t secure-cicd-demo .
08	Run Container	docker run -d -p 3000:3000 ...
09	Health Check (GATE)	docker inspect — wait until healthy
10	Stop Container	always() — clean up the container every time

After the ten steps, two notification steps run conditionally: one fires only on `success()` and the other only on `failure()`, so the team always knows the outcome.

5. Security Scanning (The Gate)

Step 4 is the heart of the DevSecOps idea. It runs `npm audit --audit-level=high`, which audits the whole dependency tree from `package-lock.json` against the npm advisory database. Any high or critical finding returns a non-zero exit code and fails the step, which in turn skips every later step — no time is wasted building or shipping code that is already known to be unsafe.

5.1 Severity Policy

Level	Exit	Pipeline behavior
low	exit 0	Informational — not gated, the pipeline keeps going.
moderate	exit 0	Tracked and planned, but does not block this pipeline.
high	exit 1	Blocked — this is our threshold, the run fails here.
critical	exit 1	Always blocked — never allowed past the gate.

5.2 Proving the Gate Works: One Dependency, Two Outcomes

To make the gate easy to demonstrate, we used two branches that pin different versions of the same library, `lodash`. The main branch pins a patched version that passes the scan, while a separate `demo/security-fail` branch pins a known-vulnerable version that is blocked.

```
# main branch – safe, passes the gate
npm install lodash@4.18.1
npm run audit:ci      # → exit 0  (pass)

# demo/security-fail – vulnerable, blocked
npm install lodash@4.17.11
npm run audit:ci      # → exit 1  (fail)
```

Opening a pull request from `demo/security-fail` makes the security scan fail in a reproducible way, which is exactly what we use for the live failure demo. The screenshots below show the audit step failing inside GitHub Actions.

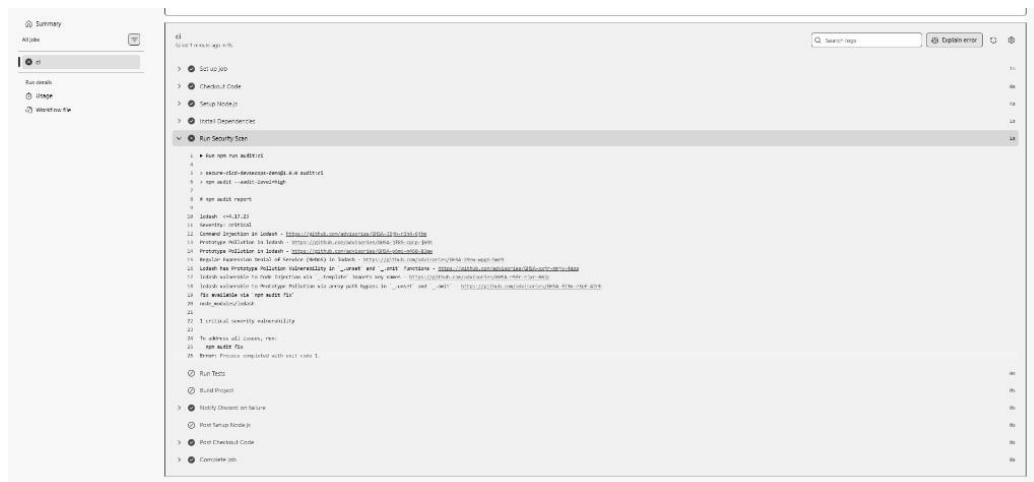


Figure 1 — The security scan failing on a known-vulnerable dependency.

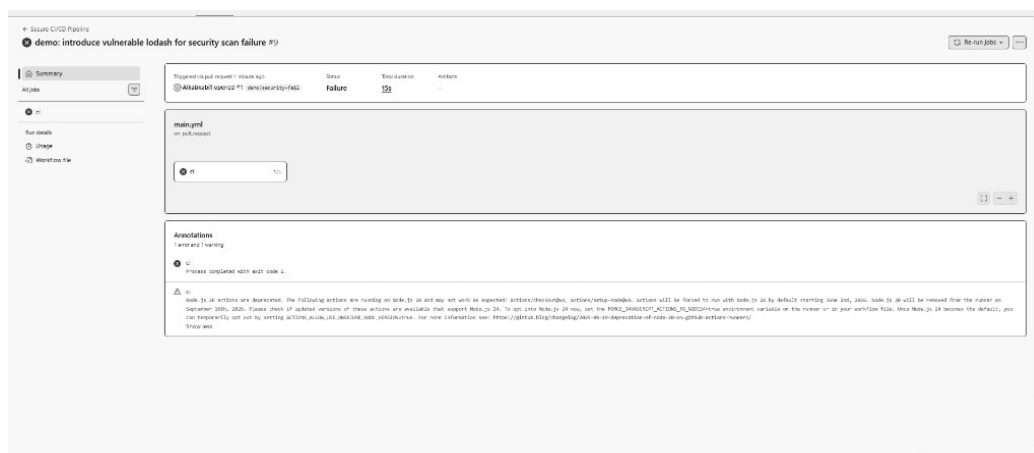


Figure 2 — npm audit summary reporting the high-severity advisory.

It is worth noting that `npm audit` only covers the npm advisory database. A real production pipeline would pair it with tools such as Dependabot, Snyk, or container image scanning to cover more classes of risk.

6. Docker and the Health Check

We containerize the app with Docker so it runs identically on the runner, a laptop, or a server — no more “it works on my machine”. The pipeline builds one image and runs that exact image, so what we test is exactly what we would deploy.

The most important idea in this part is the difference between **running** and **healthy**. A container can be running while the app inside it is dead. To catch that, the Dockerfile defines a HEALTHCHECK that repeatedly calls the app’s /health endpoint and only reports healthy once the app actually replies.

6.1 The Dockerfile

```
FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
# slim images ship without curl, so we add it
RUN apt-get update && apt-get install -y curl
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s \
  --start-period=5s --retries=3 \
  CMD curl --fail http://localhost:3000/health || exit 1
CMD ["npm", "start"]
```

The health endpoint itself is tiny — a small Node server that returns {"status": "ok"} on /health. The three retries (every 10 seconds) give the container a short grace period before it is declared failed and the pipeline reacts.

6.2 Healthy vs. Unhealthy — The Same Image, Two Outcomes

In the healthy case, we run the image as-is: the Node server boots, listens on port 3000, and the health check gets a 200 response within about ten seconds.


```

4 COPY package*.json ./
5
6 RUN npm ci
7
8 COPY . .
9
10
11 RUN apt-get update \
12     && apt-get install -y --no-install-recommends curl \
13     && rm -rf /var/lib/apt/lists/*
14
15 EXPOSE 3000
16
17 HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
18     CMD curl --fail http://localhost:3000/health || exit 1
19
20 CMD ["npm", "start"]

```

```

PS C:\Users\amjaa\secure-cicd-devsecops-demo2> docker run -d --name healthy-container -p 3000:3000 devsecops-app
404c77d2d5136df4c06da001a8cb29386e8b74bcbffed8101dc3e8b9ab365fb4
PS C:\Users\amjaa\secure-cicd-devsecops-demo2> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
404c77d2d513   devsecops-app  "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
healthy-container
PS C:\Users\amjaa\secure-cicd-devsecops-demo2>

```

Figure 3 — The container reports “healthy” after the app starts and answers /health.

In the unhealthy case, we override the start command so the container stays up but the app never boots. Every /health call fails, the retries are exhausted, and after the grace period the container is reported as unhealthy — which the pipeline treats as a failure.

```

4 COPY package*.json ./
5
6 RUN npm ci
7
8 COPY . .
9
10
11 RUN apt-get update \
12     && apt-get install -y --no-install-recommends curl \
13     && rm -rf /var/lib/apt/lists/*
14
15 EXPOSE 3000
16
17 HEALTHCHECK CMD curl --fail http://localhost:9999/health || exit 1
18
19 CMD ["npm", "start"]
20

```

```

PS C:\Users\amjaa\secure-cicd-devsecops-demo2> docker build -t broken-app .
PS C:\Users\amjaa\secure-cicd-devsecops-demo2> docker run -d --name broken-container broken-app
ce02e4fd06ae7878351a01fb45733e3030c918073342b0acf8318fd4e72709e3
PS C:\Users\amjaa\secure-cicd-devsecops-demo2> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
ce02e4fd06ae   broken-app     "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (unhealthy)  3000/tcp
broken-container
404c77d2d513   devsecops-app  "docker-entrypoint.s..." 14 minutes ago  Up 14 minutes (healthy)  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
tcp healthy-container
PS C:\Users\amjaa\secure-cicd-devsecops-demo2>

```

Figure 4 — The same image reported “unhealthy” when the app never starts.

7. Real-Time Notifications

When a run finishes, the pipeline sends an instant message to the team’s Telegram group using the Telegram Bot API. There are two message templates: a success summary, and a failure alert that carries the branch and the likely cause. The bot token and chat ID live in **GitHub Secrets** — never in the code or the logs.

Each notify step is also marked `continue-on-error: true`, so a hiccup in the notification itself can never hide the real result of the pipeline.

7.1 The Notify Step

```
- name: Notify Telegram on failure
  if: failure()
  continue-on-error: true
  env:
    TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
    TELEGRAM_CHAT_ID:  ${ secrets.TELEGRAM_CHAT_ID }
  run: |
    API="https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN"
    curl -sS -X POST "$API/sendMessage" \
      --data "$(jq -n --arg chat "$TELEGRAM_CHAT_ID" \
        --arg text "$TEXT" \
        '{chat_id: ($chat|tonumber), text: $text,
        parse_mode: "HTML"}')"
```

Building the JSON payload with `jq --arg` (instead of by hand in bash) means special characters in a commit message can no longer break the request — a small detail that caused a real bug during development, described in the next section.

8. The Live Demo

The demo runs two scenarios back to back, to show both halves of the gate behavior:

Scenario A — Success (branch: main)

1. Push to main, which uses the safe `1odash@4.18.1`.
2. All ten steps pass — scan, tests, build, Docker, and health check.
3. A  Pipeline Success message arrives on Telegram.

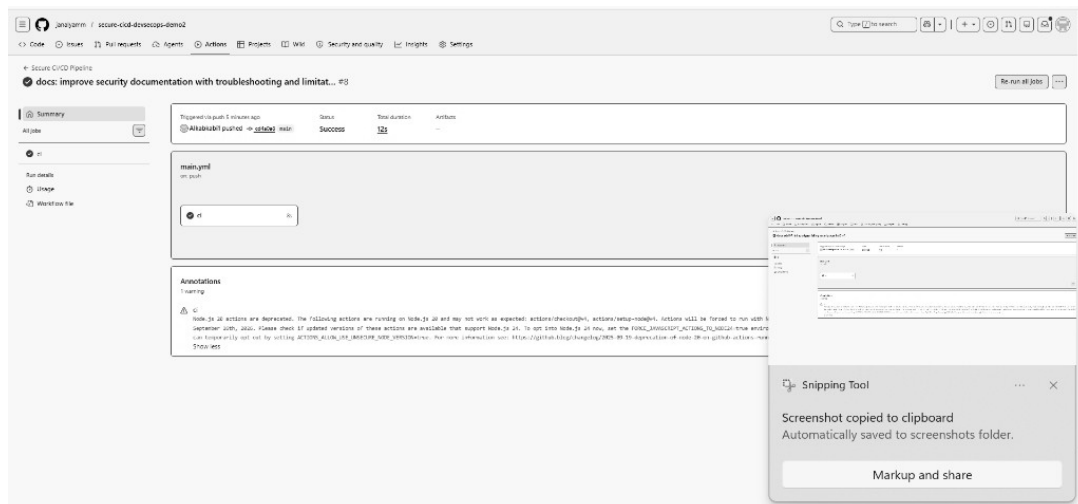


Figure 5 — A successful pipeline run on the main branch.

Scenario B — Failure (branch: demo/security-fail → main)

1. Open a pull request from demo/security-fail with lodash@4.17.11.
2. The security scan fails on a high advisory.
3. Tests, build, and Docker are skipped automatically (fail-closed).
4. A 🚫 failure alert hits Telegram instantly.

9. Challenges and Lessons Learned

Most of the real learning came from the problems we hit while building the pipeline. The most important ones, and how we solved them, are below.

01. The audit was passing on nothing

Problem: npm audit ran green because package-lock.json was not committed, so there was no dependency tree to scan.

Lesson: A lockfile is mandatory for a meaningful scan.

02. The “safe” version went unsafe

Problem: lodash@4.17.21 started failing as advisories were updated over time.

Lesson: Security is a moving target — automate updates instead of pinning a version once.

03. Step order wasted runner time

Problem: Building the Docker image before scanning burned minutes on runs that were doomed to fail.

Lesson: Fail-fast — put the cheapest, most-likely-to-fail checks first.

04. Telegram formatting broke

Problem: Special characters in commit messages broke the HTML payload we sent.

Lesson: Build the JSON with jq --arg, never by hand in bash.

05. curl missing in node:20-slim

Problem: The HEALTHCHECK always failed because curl was not in the slim image.

Lesson: Install whatever your health check depends on.

06. A token nearly leaked

Problem: A bot token was almost pasted into chat and logs.

Lesson: Secrets live only in GitHub Secrets — rotate one the moment it is exposed.

10. Conclusion

This project shows, end to end, what it means to put security inside the pipeline rather than bolting it on at the end. Every push or pull request is automatically scanned, tested, built into a Docker container, and verified to be healthy before anything is considered done — and the team is notified the moment the result is known.

The two gates, the security scan and the health check, both behave fail-closed: when something is wrong, the pipeline stops and raises an alert instead of quietly continuing. Beyond the working tutorial, the biggest takeaways were practical — a lockfile is essential for scanning, the cheapest checks should run first, secrets belong only in a secret store, and security is a moving target that has to be automated rather than solved once.

11. References

- [1] GitHub, “GitHub Actions documentation,” GitHub Docs. [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: Jun. 3, 2026].
- [2] npm, Inc., “npm-audit,” npm Docs. [Online]. Available: <https://docs.npmjs.com/cli/commands/npm-audit>. [Accessed: Jun. 3, 2026].
- [3] Docker, Inc., “Dockerfile reference: HEALTHCHECK instruction,” Docker Documentation. [Online]. Available: <https://docs.docker.com/engine/reference/builder/#healthcheck>. [Accessed: Jun. 3, 2026].
- [4] Telegram, “Telegram Bot API,” Telegram Core. [Online]. Available: <https://core.telegram.org/bots/api>. [Accessed: Jun. 3, 2026].
- [5] OWASP Foundation, “OWASP DevSecOps guideline,” OWASP. [Online]. Available: <https://owasp.org/www-project-devsecops-guideline/>. [Accessed: Jun. 3, 2026].
- [6] Node.js, “Node.js documentation,” Node.js. [Online]. Available: <https://nodejs.org/en/docs>. [Accessed: Jun. 3, 2026].